**Project 2: Linux Systems Programming**

**Repository :** Project-2_Assignment_linuxProgramming

**Date :** 17th July 2026

| Name | Link |
|---|---|
| Question 1: fork, execvp , pipe | child_processes.c |
| Question 2 : low-level system calls \| standard I/O functions | copy_system.c |
| | copy_stdio.c |
| Question 3: prime numbers between 1 and 200,000 using 16 threads | multithreading.c |
| Question 4: multithreaded keyword search across files | search.c |

### Question 1 : process pipeline

The objective of this assignment was to help me understand how processes communicate in Linux using system calls. Since I am learning process programming for the first time, I wanted to understand what happens behind the scenes when a command like ps aux | grep root is executed. Instead of running the command directly in the terminal, I wrote a C program that creates the pipeline using fork(), pipe(), dup2(), and execvp(). I also used **strace** to observe the system calls used during execution.

### Implementation

I first created a pipe so that two child processes could communicate. The first child process redirects its standard output to the pipe and executes ps aux using execvp(). The second child redirects its standard input to the pipe, redirects its standard output

to **output.txt**, and executes grep root. This allows the output of ps aux to be filtered and saved into the file. The parent process closes the pipe, waits for both children to finish using wait(), then opens the file, reads part of its contents, and displays it on the screen.

**Challenges**

The main challenge was understanding how dup2() redirects input and output. At first, I found it confusing how one process writes to the pipe while another reads from it. I also learned that every process must close the pipe ends it does not use. Another challenge was making sure the parent waited for both child processes before reading the output file, so that all the data had already been written.

**Execution Behavior**

The program compiled and ran successfully. It created two child processes, executed ps aux and grep root, saved the filtered output into **output.txt**, and displayed part of the file. Using **strace**, I confirmed that the expected system calls were executed, including fork() (shown as clone()), pipe() (pipe2()), dup2(), execve(), open(), read(), write(), close(), and wait4(). This helped me understand how Linux creates processes, redirects input and output, and manages communication between processes.

gcc -o child_processes child_processes.c

./child_processes

Runs ps aux | grep root internally via two forked children connected by a pipe, writes the filtered result to output.txt, then the parent reads and prints a preview.

## Question 2: File Copy Using System Calls and Standard I/O

The goal of this question was to compare two different ways of copying a large file in C. The first version uses low-level Linux system calls (read() and write()), while the second version uses the standard C library functions (fread() and fwrite()). Since I am learning file handling for the first time, this assignment helped me understand the difference between these two approaches. I also used **strace** to compare the number of system calls and measured the execution time of each program while copying a 100 MB file.

### Implementation

I wrote two separate programs because each one demonstrates a different method of copying a file. The first program, **copy_system.c**, opens the source and destination files using open(), copies the file with read() and write(), and closes the files using close(). The second program, **copy_stdio.c**, performs the same task using fopen(), fread(), fwrite(), and fclose(). Both programs use the same buffer size of **4096 bytes** and measure their execution time using clock().

### Challenges

The main challenge was understanding why I needed two separate programs instead of combining everything into one file. I learned that the purpose of the assignment is to compare two different file-copy methods fairly. Another challenge was understanding why the strace results showed almost the same number of system calls. After reviewing the results, I realized that both programs use the same 4096-byte buffer, so they make almost the same number of read() operations. This showed me that using fread() and fwrite() does not always reduce the number of system calls. The difference depends on the buffer size being used.

**Execution Behavior**

gcc -o copy_system copy_system.c

gcc -o copy_stdio copy_stdio.c

Needs a large source file named largefile.bin in the working directory:

dd if=/dev/urandom of=largefile.bin bs=1M count=100

./copy_system # writes copy_system.bin

./copy_stdio # writes copy_stdio.bin

Both programs successfully copied the 100 MB file, and I confirmed that the copied files were identical to the original using diff. The execution time showed that the standard I/O version was slightly faster (**0.1932 seconds**) than the system call version (**0.2185 seconds**). Using **strace**, I also observed that both programs made nearly the same number of system calls because they used the same buffer size. From this assignment, I learned that standard I/O can improve performance, but it does not always reduce the number of system calls. The results depend on how the program is implemented and the buffer size that is used.

**Question 3: Parallel prime counting (pthread, mutex)**

The objective of this assignment was to learn how to use multiple threads in C to solve a problem faster. Since I am learning multithreading for the first time, this assignment helped me understand how POSIX threads (pthread) work, how a large task can be divided among several threads, and why synchronization is important when multiple threads access the same shared variable.

**Implementation**

I created a program that uses **16 POSIX threads** to count the prime numbers between **1 and 200,000**. I divided the range equally so that each thread processes its own section of numbers. Each thread counts the prime numbers in its assigned range and stores the result in a local variable. After finishing, the thread updates the shared variable totalPrimes. To prevent multiple threads from modifying the shared variable at the same time, I used a pthread_mutex_t mutex. Each thread locks the mutex before updating the counter and unlocks it immediately after the update. Finally, the main thread waits for all 16 threads to finish using pthread_join() before printing the total number of prime numbers.

**Challenges**

The main challenge was understanding why a mutex was necessary. At first, I thought each thread could update the shared counter directly. However, I learned that if multiple threads write to the same variable at the same time, the final result may be incorrect due to a race condition. Using pthread_mutex_lock() and pthread_mutex_unlock() ensured that only one thread updated the shared counter at a time. I also had to make sure that the workload was divided evenly so every thread processed a different range of numbers.

**Execution Behavior**

./multithreading

No arguments; always counts primes in [1, 200000] across 16 threads.

gcc -O2 -pthread -o multithreading multithreading.c

The program compiled and executed successfully. All **16 threads** were created, each processed its assigned range, and the final result was printed only after all threads completed their work. The output displayed the synchronized total number of prime numbers between **1 and 200,000**, confirming that the threads worked correctly and that the mutex successfully protected the shared counter from race conditions. This assignment helped me understand how multithreading can improve performance while synchronization ensures that shared data remains correct.

**Question 4: Multithreaded file search (pthread, mutex, CLI args)**

The goal was to search for a keyword across multiple text files in parallel, one thread per file, writing all results to a single shared output file, and to compare execution time across different thread counts.

**Implementation**

For this one I wrote a program that accepts a keyword, an output file, one or more input files, and the number of threads as command-line arguments. Each thread is assigned one file to search. The thread opens its file using fopen(), reads each word with fscanf(), compares it with the keyword using strcmp(), and counts the number of matches. After finishing, the thread writes its result to a shared output file using fprintf(). Since multiple threads share the same output file, I used a pthread_mutex_t mutex to ensure that only one thread writes to the file at a time. The program measures the execution time using clock() and repeats the test with different thread counts.

**Measured results**

./search <keyword> <output.txt> <file1.txt> <file2.txt> ... <number_of_threads>

For example :

./search root results.txt logs/a.txt logs/b.txt logs/c.txt 4

gcc -O2 -pthread -o search search.c

For testing, I used six text files, each around 128 KB and containing about 2,000 occurrences of the keyword "root." I then ran the program using the three thread configurations required by the assignment and compared the execution times:

| Threads requested | Why this number | Threads actually used | Execution time |
|---|---|---|---|

| | | | |
|---|---|---|---|
| 2 | The 2-thread case the assignment asks for | 2 | 0.0172 s |
| 6 | Max threads one thread per file, since there are 6 files | 6 | 0.0219 s |
| 8 | Average CPU cores nproc reports 8 cores on this machine | 6 (capped, only 6 files to hand out) | 0.0187 s |

All three runs gave the exact same occurrence counts per file, so asking for more threads never changed the answer, only the speed. With files this small, the actual search barely takes any time, so most of what you're measuring is the overhead of creating and joining threads  that's why 6 threads isn't clearly faster than 2, and why asking for 8 threads doesn't help once it gets capped down to 6. You'd need much bigger files before adding threads actually pays off.

**Challenges**

The main challenge was understanding how to safely write to one output file from multiple threads. Without synchronization, two threads could write at the same time and produce incorrect or mixed output. I solved this by protecting the writing section with pthread_mutex_lock() and pthread_mutex_unlock(). Another challenge was handling cases where the requested number of threads was greater than the number of input files. I solved this by limiting the number of threads to the number of available files so that every thread had useful work to do.